

CSCE 658 RANDOMIZED ALGORITHMS · MODULE 01 · LECTURE NOTES

Why Randomness

Three distinct things a coin buys.

MODULE OUTCOMES

- Distinguish the three uses of randomness and give examples.
- Distinguish Las Vegas from Monte Carlo algorithms.
- Explain why a probabilistic guarantee is a guarantee.
- Explain the adversary model and what randomness does to it.
- Decide whether randomness is the right tool for a problem.

1 The three uses

Use	The mechanism	Examples
Defeat an adversary	The input cannot be chosen against choices the adversary cannot see.	Universal hashing, randomised quicksort pivots, cuckoo hashing, randomised load balancing. See the callout.
Simplify the algorithm	Expected balance instead of explicitly enforced balance.	Skip lists against balanced trees, treaps against red-black trees, Karger's random contraction against deterministic min-cut. The randomised versions are dramatically shorter.
Sample instead of computing	Estimate a quantity from a subset rather than examining everything.	Streaming sketches (Module 05), Monte Carlo integration (CSCE 647), stochastic gradient descent (CSCE 669 Module 08).

The alternatives differ, which is why the three are worth separating: the first can be replaced by a secret key (and nothing else), the second by a more complex deterministic structure (which exists and is longer), and the third frequently cannot be replaced at all — if the data does not fit in memory, no deterministic algorithm examines all of it either. So 'can I derandomise this?' has three different answers depending on which use you are making (Module 12).

DEFEATING AN ADVERSARY IS THE USE THAT CANNOT BE REPLACED BY EFFORT

A deterministic hash function has a worst-case input — whatever set of keys happens to collide. And an adversary who knows your hash function can construct that input, which is exactly how hash-flooding denial-of-service attacks work and why language runtimes now randomise their string hashing.

Randomising the choice of hash function means the adversary must defeat a function they cannot see — the worst-case input still exists, and they cannot find it.

So the guarantee changes from 'no bad input exists' (which is false) to 'a bad input is unlikely to be hit' (which is true and quantifiable) — and the second guarantee is the one you can actually have.

And this is the use where randomness is irreplaceable by cleverness. No amount of deterministic ingenuity removes a worst case that an adversary can compute, because the adversary gets to see your algorithm. Only a secret helps, and a random choice is the cheapest available secret — which is also why this use is the one that overlaps with cryptography (CSCE 711).

2 Las Vegas and Monte Carlo

LAS VEGAS IS ALWAYS RIGHT; MONTE CARLO IS USUALLY RIGHT

Las Vegas: the answer is always correct, and the *running time* is a random variable. Randomised quicksort, treaps, the construction phase of perfect hashing.

Monte Carlo: the running time is bounded, and the *answer* may be wrong. Miller–Rabin primality, Bloom filters, count-min sketches, almost all of Module 05.

And the difference decides what you must do about it. A Las Vegas algorithm can simply be re-run when it takes too long — you can see that it has — whereas a Monte Carlo failure is silent unless you have an independent way to verify the answer.

So prefer Las Vegas whenever the choice exists, and when using Monte Carlo, establish which direction the error goes — a Bloom filter has false positives and never false negatives, which is what makes it usable as a pre-filter, and a structure with the opposite asymmetry would be useless for that purpose. One-sided error is far easier to design around, and knowing which side you have is the first question.

	Las Vegas	Monte Carlo
What to report	Expected running time <i>and</i> a tail bound.	The failure probability, and the direction of the error.
What a failure is	It took longer than expected. Re-run it.	A wrong answer, possibly silent.
Detectable?	Yes — by the clock.	Only if you can independently verify the answer, which for primality you cannot cheaply and for a found factor you can.

	Las Vegas	Monte Carlo
How to amplify	Re-run on a timeout, which bounds the tail.	Repetition and majority vote, if the error is two-sided; repetition alone if one-sided (Module 02 §4).
Canonical example	Randomised quicksort.	Bloom filter, count-min sketch.

3 Why a probabilistic guarantee is a guarantee

2^{-40} IS BELOW THE HARDWARE ERROR RATE

A randomised algorithm with failure probability 2^{-40} fails roughly once in a trillion runs. At a thousand runs per second it fails about once every thirty thousand years.

Measured uncorrected DRAM bit-flip rates are on the order of one error per gigabyte per month, and a long deterministic computation touches a great deal of memory for a long time. So its probability of producing a wrong answer is not zero either, and is frequently larger.

So the randomised algorithm is more reliable than the machine running the deterministic one — which answers the objection that randomised algorithms 'might be wrong' completely rather than partially. Everything might be wrong; the question is the probability, and only one of the two has a bound you can compute.

And the amplification is cheap: repetition drives the failure probability down exponentially (Module 02 §4, CSCE 637 Module 08 §1), so the exponent is a design parameter rather than a fixed property of the algorithm — you choose how reliable you want it to be, and pay linearly for an exponential improvement. That trade is the best one in this course.

4 When randomness is the wrong choice

- When reproducibility is required and seeding is impractical. A randomised build process or a compiler whose output varies between runs is a real operational problem, and reproducible builds are a security property — so determinism is sometimes the feature (Module 13 §1).
- When the deterministic algorithm is genuinely simpler. It frequently is not — a skip list against a red-black tree is the standard illustration — but when it is, take it.
- When you cannot get good randomness. An embedded device at first boot has essentially no entropy, and a weak source voids the guarantee entirely rather than degrading it (Module 13 §2) — which has caused real and serious failures.
- When the cost of a failure is unbounded. 2^{-40} is fine for a cache lookup and not for an irreversible financial or physical action, whatever §3's arithmetic says — because the arithmetic multiplies probability by cost and one of the two may not be finite.
- And when the guarantee is about the average and the requirement is about the tail. Which is the commonest misuse (Module 02 §1): an expected-time guarantee says nothing useful about a 99th-percentile latency objective (CSCE 678 Module 06), and the two are routinely conflated by people who have read the expected-case analysis and stopped.

WHERE THIS COURSE GOES

Module 02 is the toolkit — Markov, Chebyshev, Chernoff, and the union bound. **It is the whole course in one module, and the one to know cold:** almost every subsequent analysis is one of those four applied to a well-chosen random variable.

Modules 03 through 06 are the classical analyses. Balls and bins, randomised data structures, sketches and streaming, and random graphs — **which is to say, the things you have already used without analysing.**

Modules 07 through 10 are randomness as a method. The probabilistic method as a proof technique, randomised rounding for approximation, Markov chains and mixing, and dimension reduction.

Modules 11 through 13 are randomness in systems, derandomisation, and honesty about the randomness itself — and the last of those is the one with the most practical consequence.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mitzenmacher & Upfal — Probability and Computing, chapter 1	The three uses and the two kinds of algorithm , with the verifying-matrix-multiplication example that motivates §2.
Motwani & Raghavan — Randomized Algorithms, chapter 1	The Las Vegas / Monte Carlo distinction and the adversary framing of §1.
Crosby & Wallach — Denial of Service via Algorithmic Complexity Attacks (free)	§1's adversary use, as an attack — and the paper that caused language runtimes to randomise their hashing.
Schroeder, Pinheiro & Weber — DRAM Errors in the Wild (free)	§3's hardware error rates, measured at scale — which is what makes the argument quantitative rather than rhetorical.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Classify five randomised algorithms you have used** by which of the three uses they make.
- 2 **For each, say what the deterministic alternative is** and whether one exists.
- 3 **Construct a hash-collision attack** against a deterministic hash function, and measure the slowdown.
- 4 **Randomise the hash choice** and confirm the attack fails.

- 5 **Classify five algorithms as Las Vegas or Monte Carlo**, and for the Monte Carlo ones state the error direction.
- 6 **Implement randomised quicksort** and plot its running-time distribution over many runs.
- 7 **Compute how many repetitions of a 1/3-error algorithm** give failure below 2^{-40} .
- 8 **Find the uncorrected error rate** for a machine you use, and compare.
- 9 **Find a case where a randomised algorithm was the wrong choice** — reproducibility, entropy, or cost — and describe it.
- 10 **Write down what you expect randomness to be good for**, and keep it for Module 13.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the three uses of randomness and an example of each.
- 2 How do their deterministic alternatives differ?
- 3 Why is the adversary use irreplaceable by cleverness?
- 4 Distinguish Las Vegas from Monte Carlo on five axes.
- 5 Why does the error direction matter for Monte Carlo?
- 6 Give the hardware-error argument with numbers.
- 7 Why is the failure exponent a design parameter?
- 8 Name five situations where randomness is the wrong choice.
- 9 What is the commonest misuse of a randomised guarantee?